

B29: interactive RFI explorer

Point at a range of campaign files and an antenna, and render the same 3-panel view already established in `dpss_smooth_model_and_residual_pca.ipynb` / the full-campaign checkpoint -- **data (log color) | data - DPSS model (residual) | flags/v2 mask x residual (kept-only)** -- generalized to work across an arbitrary file range instead of one fixed pilot day.

Run this cell-by-cell top to bottom, then use the controls: pick a day, drag the range slider to a start/end file within that day, pick an antenna, click Render.

What you need

Python packages	<code>numpy</code> , <code>matplotlib</code> , <code>h5py</code> , <code>ipywidgets</code> (JupyterLab/Notebook), <code>eigsep_data</code> (must be installed -- see <code>marjum-2026-07/flagging/rfi_explorer_core.py</code> 's own note on this new coupling)
Data	the real campaign tree at <code>MARJUM_DATA_ROOT</code> (defaults to <code>EIGSEP_ROOT/marjum-2026-07</code>) -- <code>data/*.h5</code> , <code>derived/smooth_model/v0/*.h5</code> (B16 fit), <code>flags/v2/*.h5</code> (mask)
Repo layout	this notebook lives in <code>data-analysis/</code> ; its backing code (<code>rfi_explorer_core.py</code>) stays in the sibling <code>eigsep / marjum-2026-07/flagging/</code> checkout -- set <code>EIGSEP_ROOT</code> if that checkout isn't at <code>/mnt/data02/eigsep</code>

Antenna resolution is transparent, not literal: "gnd" tries raw key `0` first, falling back to `3` if `0` isn't live for a given file (same physical box-gnd, a different SNAP input during campaign Phase A(late)/B). "air" is always raw key `4`. A file with neither live for the selected antenna is silently skipped from the range (reported in the used/skipped counts below the plot), not an error.

Known limitation, stated plainly rather than discovered later: this notebook's live-widget behavior (dragging the range slider, clicking Render, watching the plot update) can only be verified by a human actually running it in Jupyter. A static, re-executed export (like the PDF this checkpoint is delivered as) can only show the

plot at whatever selection was rendered before export -- it cannot demonstrate the interaction itself. What is verified here, the same way as every other deliverable this week: the underlying data-loading and plotting functions, executed for real against real data, producing a real, checkable static snapshot below.

```
In [1]: import sys, os
EIGSEP_ROOT = os.environ.get("EIGSEP_ROOT", "/mnt/data02/eigsep")
sys.path.insert(0, os.path.join(EIGSEP_ROOT, "marjum-2026-07", "flagging"))
import numpy as np
import matplotlib.pyplot as plt
import ipywidgets as W
from IPython.display import display

import rfi_explorer_core as RC

ALL_FILES = RC.list_files()
print(f"{len(ALL_FILES)} campaign files available, "
      f"{ALL_FILES[0]} .. {ALL_FILES[-1]}")

DAYS = sorted(set(f[5:13] for f in ALL_FILES))
print("days:", DAYS)
```

```
5120 campaign files available, corr_20260712_133146Z.h5 .. corr_20260718_032248Z.h5
days: ['20260712', '20260713', '20260714', '20260715', '20260716', '20260717', '20260718']
```

Controls

```
In [2]: def files_for_day(day):
        return [f for f in ALL_FILES if f[5:13] == day]

def label_for(fname):
    # HH:MM:SS from the filename's own close-time encoding
    hhmmss = fname[14:20]
    return f"{hhmmss[:2]}:{hhmmss[2:4]}:{hhmmss[4:6]} ({fname})"

day_w = W Dropdown(options=DAYS, value=DAYS[-1], description="day")
```

```

_day_files = files_for_day(day_w.value)
range_w = W.SelectionRangeSlider(
    options=[(label_for(f), f) for f in _day_files],
    index=(0, min(9, len(_day_files) - 1)),
    description="range", layout=W.Layout(width="700px"),
)
antenna_w = W.Dropdown(options=[("box-gnd", "gnd"), ("box-air", "air")],
    value="air", description="antenna")
render_btn = W.Button(description="Render", button_style="primary")
out = W.Output()

def on_day_change(change):
    day_files = files_for_day(day_w.value)
    range_w.options = [(label_for(f), f) for f in day_files]
    range_w.index = (0, min(9, len(day_files) - 1))

day_w.observe(on_day_change, names="value")

def on_render_click(_b=None):
    start_fname, end_fname = range_w.value
    antenna = antenna_w.value
    with out:
        out.clear_output(wait=True)
        try:
            result = RC.load_range(start_fname, end_fname, antenna, ALL_FILES)
        except ValueError as e:
            print(f"Nothing to show: {e}")
            return
        fig = RC.plot_three_panel(
            result, antenna,
            title_extra=f", {start_fname}..{end_fname}")
        plt.show()
        print(f"used {len(result['used_files'])} files, "
            f"skipped {len(result['skipped_files'])} "
            f"(no live '{antenna}' data or no B16 companion)")

```

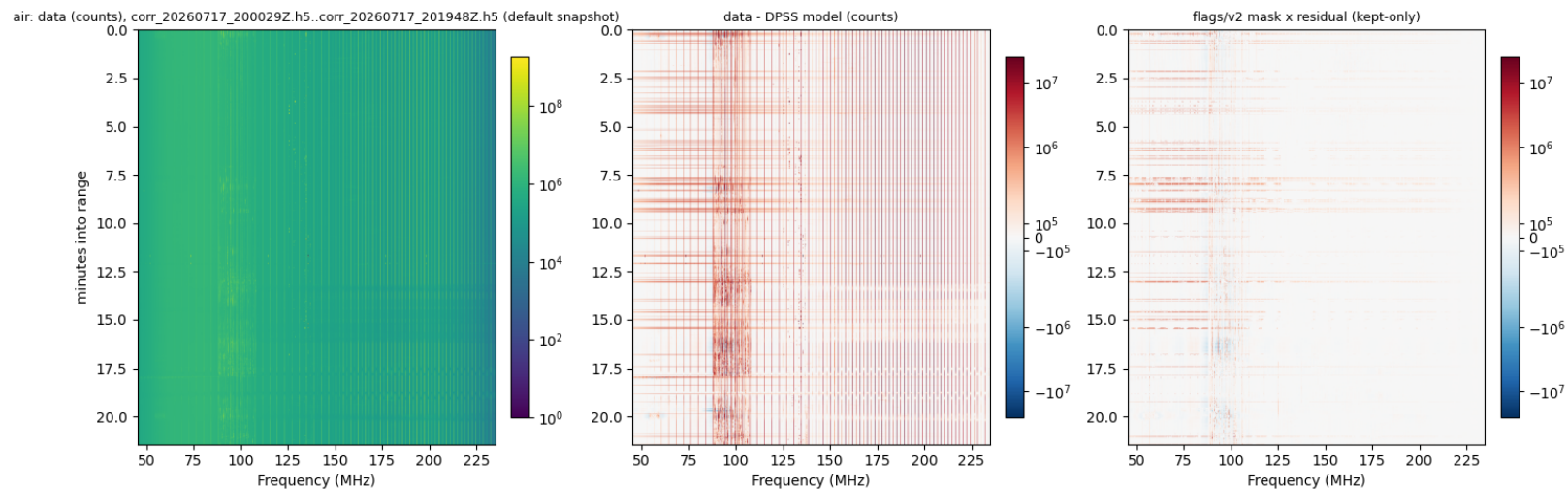
```
render_btn.on_click(on_render_click)

display(W.VBox([W.HBox([day_w, antenna_w]), range_w, render_btn, out]))
```

Static default-state snapshot (this is what a re-executed export can show)

Same range/antenna as the pilot notebook's own pilot window, rendered directly (not via the button, so this cell always produces real output when the notebook is re-executed non-interactively).

```
In [3]: default_start, default_end = "corr_20260717_200029Z.h5", "corr_20260717_201948Z.h5"
default_antenna = "air"
result = RC.load_range(default_start, default_end, default_antenna, ALL_FILES)
fig = RC.plot_three_panel(result, default_antenna,
                          title_extra=f", {default_start}..{default_end} (default
snapshot)")
plt.show()
print(f"used {len(result['used_files'])} files, skipped {len(result['skipped_files'])}")
```



used 10 files, skipped 0

Reading the panels

Same convention as the full-campaign checkpoint and the DPSS pilot notebook: **panel 1** (data, log color) shows the raw band shape and every comb/RFI feature at full contrast; **panel 2** (data - DPSS model) is the residual the smooth-band fit leaves behind; **panel 3** (flags/v2 mask x residual, kept-only) blanks anything v0's category bits, the static self-comb exclusion, or B16's DPSS-residual-outlier bit called bad, so what remains is what the pipeline currently believes is trustworthy signal.

A file gets silently skipped from a range if the selected antenna has no live raw data for it, or if it has no B16 companion in `derived/smooth_model/v0/` (not yet fit, or genuinely all-zero for that antenna -- see this week's B16 full-campaign checkpoint for the full accounting of why files go missing). The used/skipped counts below each render report this directly rather than silently narrowing the range.

STOPPED AT REVIEW GATE — awaiting Aaron's approval.